به نام خدا

محمد کیوانی فر (40231421) جواب سوالات پروژه دوم سیستم های نهفته و بی درنگ

پاسخ سوال اول :

بدون استفاده از یک سیستم‌عامل بلادرنگ، استفاده از تابع ()delay به دلیل مسدود کردن کامل پردازنده (Busy-waiting) عملاً کارایی ندارد و مانع اجرای همزمان سایر تسک‌ها با زمان‌بندی دقیق می‌شود .برای پیاده‌سازی این پروژه به روش سنتی، باید از روش غیرمسدودکننده (Non-blocking) مبتنی بر تایمر استفاده کنیم :

1. با استفاده از تابع ()millis، زمان سپری‌شده سیستم را مدام رصد می‌کنیم.
2. برای هر LED یک متغیر زمانِ قبلی (previousMillis) تعریف می‌کنیم.
3. در داخل حلقه اصلی ()loop، در هر لحظه اختلاف زمان فعلی را با زمان قبلی هر سه تسک مقایسه می‌کنیم :

(currentMillis - previousMillis >= period)

4. در صورت رسیدن به بازه زمانی موردنظر (۱۰۰، ۵۰۰ یا ۱۰۰۰ میلی‌ثانیه)، وضعیت LED مربوطه را معکوس (Toggle) کرده و زمان قبلی آن را به‌روزرسانی می‌کنیم.

پاسخ سوال دوم :

خیر مشکل این تابع این است که تابع vTaskDelayتاخیر را نسبت به لحظه فراخوانی خودش اعمال می‌کند .یعنی اگر اجرای کدهای داخل تسک (مثلاً دستورات تغییر وضعیت پین) کمی طول بکشد یا زمان‌بند (Scheduler) به دلیل اجرای تسک‌های هم‌اولویت یا با اولویت بالاتر، برای مدتی اجرای این تسک را به تعویق بیندازد، این زمانِ تلف‌شده به دوره تناوب اضافه می‌شود. در نتیجه، در طول زمان پدیده رانش زمان (Time Drift) رخ می‌دهد و تسک رفتار پریودیک دقیق خود را از دست می‌دهد .

در باره ی vTaskDelayUntil :

تابع vTaskDelayUntil برای پیاده‌سازی تسک‌های پریودیک کاملاً دقیق و ثابت طراحی شده است. تفاوت اصلی آن با vTaskDelay در این است که تاخیر را نسبت به یک زمان مرجع مطلق (Absolute Time) حساب می‌کند، نه نسبت به لحظه فراخوانی تابع. این تابع زمان شروع آخرین اجرای تسک را به عنوان ورودی می‌گیرد و دقیقاً به اندازه دوره تناوب (مثلاً ۵۰۰ تیک) به آن اضافه می‌کند. به این ترتیب، حتی اگر اجرای کدهای داخلی تسک یا جابجایی بین تسک‌ها زمان ببرد، زمان‌بند سیستم‌عامل تضمین می‌کند که تسک بعدی دقیقاً در همان پریودِ ثابتِ تعریف‌شده از حالت Blocked خارج شود و مشکل رانش زمان (Drift) کاملاً حل می‌شود.